AI 汇编语言：面向端侧模型（Edge LLMs）的 16 指令操作系统控制集

AI Assembly Language: A 16-Instruction Operating System Control Set for Edge LLMs

Name: Constant-Dalink
Location: Hubei, China
Contact: 13581432693@163.com / 13581432693
Independent Researcher

Preprint v1.0 — 2026

摘要

现有 LLM Agent 框架在桌面控制场景中，通常要求底层模型具备较强的结构化输出能力、长上下文处理能力，或视觉多模态能力。因此，相关系统往往依赖 GPT-4 级别的大模型，或依赖成本较高的云端调用与多模态推理。这使桌面自动化在延迟、成本、隐私和离线可用性方面受到一定限制。

本文提出一种以表示压缩为核心的工程路径：**不增加模型复杂度，而是降低操作表示复杂度**。

具体而言，本文给出 GSS-OS 的完整 **16 条原子指令集**。该指令集可以视为一种面向操作系统控制的“AI 汇编语言”：每条指令在语义上尽量保持原子化，在文本上尽量简短，在组合上覆盖常见的人机物理交互渠道。通过将桌面控制表示为这 16 条指令的有限序列，系统可将操作任务转写为端侧小模型更容易处理的短文本生成问题。

本文的主要工作包括：(1) 给出 GSS 16 条指令的完整规格，包括语法、参数约束、执行语义与工程映射；(2) 提出指令设计的七项原则，用以说明该指令集的构成依据；(3) 引入 Constant-Dalink 不完备公式，解释指令集规模与认知经济性之间的关系；(4) 给出面向 7B 模型与 GPT-4 的实验设计与性能预期模型；(5) 讨论该指令集在端侧部署、离线运行、隐私保护与低延迟控制中的工程意义。

本文的目标不是构造一个适用于所有任务的统一 Agent 表示，而是为操作系统控制这一特定任务域给出一种更适合小模型的动作语言。

关键词：Edge AI；指令集架构；LLM Agent；桌面自动化；Token 经济学；小模型；操作系统控制

1. 引言

1.1. 研究背景

当前 AI 桌面控制系统通常建立在一个隐含前提之上：要让模型稳定地完成桌面任务，需要较强的基础模型能力。

这一判断有其工程背景。在常见框架中，模型往往需要完成以下任务中的若干项：

- 输出结构化 JSON；
- 处理较长的工具定义和上下文；
- 执行多步规划；
- 解析截图或多模态输入。

在这些条件下，小模型的表现通常不如大模型，因此许多桌面 Agent 系统默认采用 GPT-4、Claude 3.5 或其他高能力模型作为底座。

这一技术路径通常带来以下影响：

- 需要稳定联网调用云端 API；
- 每次操作都产生推理成本；
- 屏幕内容、文件名与操作序列可能进入外部服务；
- 延迟受网络状态影响；
- 离线场景受限。

1.2. 基本观点

本文的基本观点是：

> 在桌面控制任务中，小模型的限制不仅来自模型本身，也来自任务表示方式。

如果给一个 7B 模型较长的 JSON Schema、复杂的输出结构和较大的搜索空间，它更容易在格式或结构上出错。但如果只要求它在一个较小的动作词表中输出诸如 `c D4`、`hk Ctrl+S` 这样的短文本序列，问题的难度会明显下降。

因此，性能不完全由模型规模决定，还取决于：

1. 系统要求模型输出什么形式；

2. 系统向模型提供多大的上下文负担；
3. 每一步动作决策的候选空间有多大。

GSS 的 16 条指令，正是围绕这三个方面进行压缩而设计的。

1.3. 与精简指令集的类比

这一思路与经典计算机体系结构中的精简指令集思想具有一定相似性。复杂指令集倾向于将更多任务封装到单条指令中；精简指令集则使用更少、更稳定的基本指令，通过组合完成复杂行为。

GSS 的设计更接近后一种做法。它不定义“打开 Word 并另存为 PDF”这样的宏动作，而是将其拆分为若干原子步骤，例如：

- `o Word`
- `hk Ctrl+Shift+S`
- `t report.pdf`
- `hk Enter`

在这种设计下，复杂性主要由动作序列承担，而不是由指令集本身扩张承担。

1.4. 与前三篇工作的关系

本文是 GSS 系列中偏工程化的一篇，重点是给出 16 条指令集的完整规格。各篇工作的关系如表 1 所示。

表 1: GSS 系列论文关系

论文	主要内容	本文的承接
论文一	形式化定义与概率模型	本文给出指令集的工程规格
论文二	Grid Domain 视觉降维	本文说明鼠标类指令与 Grid 的配合方式
论文三	HOW-code 安全进化协议	本文说明 `hc`、`mk_lib`、`mv` 的指令层语义
本文	16 条指令集与端侧部署	—

2. 指令设计的七项原则

在给出指令集之前，需要先说明其设计约束。GSS 的 16 条指令并非随意罗列，而是基于以下七项原则确定。

2.1. 原子性（Atomicity）

每条指令应当对应一个尽可能不可再分的最小操作单元。例如：

- `o notepad` 与 `t hello` 分别表示启动程序与输入文本；
- `open_and_type("notepad","hello")` 则把两个不同动作混在了一条指令中，因此不满足原子性。

2.2. 正交性（Orthogonality）

不同指令之间应尽量减少功能重叠。理想情况下，一种基础动作只由一种原子指令承担，以避免模型在多个等价表示之间摇摆。

2.3. 覆盖性（Coverage）

指令集的组合应覆盖常见的人机操作通道。也就是说，桌面交互中常见的键盘、鼠标、窗口、进程与基本控制操作，应能由这些原子指令的有限序列表达。

2.4. 极短性（Brevity）

每条指令的文本形式应尽量简短。操作码一般控制在 1–6 个字符，参数形式也应尽量短，以使单条指令保持较低 Token 成本。

2.5. 可预测性（Predictability）

指令形式应尽量接近 LLM 的高概率续写模式。例如使用自然缩写：

- `c` 表示 click；
- `hk` 表示 hotkey；
- `t` 表示 type。

而不使用不透明编码方式。

2.6. 快捷键优先（Hotkey-First）

凡是能够稳定通过快捷键完成的操作，优先使用快捷键，而不是鼠标定位。原因在于快捷键通常具有更强的确定性和更低的空间依赖。

2.7. 认知经济性（Cognitive Economy）

根据 Constant-Dalink 不完备公式：

$$\forall \pm \{(V_1, V_2)^c, V_1, V_2, V, \dots\} \xrightarrow{\text{Check}} \circ V_{use}$$

运行时常驻的指令定义本身就是模型当前需要加载的知识子集之一。因此，指令集必须足够小，使其完整定义能够在较小 Token 预算内呈现。

这七项原则共同约束了 GSS 指令集的规模与构成。

3. GSS v1.0 的 16 条指令规格

3.1. 总览

按功能域划分，GSS v1.0 的 16 条指令如表 2 所示。

表 2: GSS v1.0 指令域划分

域	指令	数量
鼠标操作	`c`, `dc`, `rc`, `sc`, `dr`	5
键盘操作	`hk`, `t`	2
进程管理	`o`, `k`, `w`	3
流控制	`sl`	1
知识管理	`ld`	1
自进化	`hc`, `mk_lib`, `mv`	3
通信	`msg`	1
合计		16

3.2. 逐条规格

指令 01：c（Click）

字段	内容
操作码	c
全称	Left Click
语法	c <grid_ref>
参数	网格坐标代号
执行语义	将鼠标移动至目标坐标中心并执行单次左键点击
Token 消耗	2–3
示例	c D4
底层调用	pyautogui.click(x, y)
前置条件	当前网格层级已设定
副作用	鼠标位置变化，并可能触发 UI 事件

指令 02：dc（Double Click）

字段	内容
操作码	dc
全称	Double Click
语法	dc <grid_ref>
参数	网格坐标代号
执行语义	在目标位置执行双击
Token 消耗	2–3
示例	dc B2
底层调用	pyautogui.doubleClick(x, y)
说明	双击具有时间窗口要求，不能简单等同于两个普通 c

指令 03：rc（Right Click）

字段	内容
操作码	rc
全称	Right Click
语法	rc <grid_ref>
参数	网格坐标代号
执行语义	在目标位置执行右键点击
Token 消耗	2–3
示例	rc F5
底层调用	pyautogui.rightClick(x, y)

指令 04：sc（Scroll）

字段	内容
操作码	sc
全称	Scroll
语法	sc <direction> <amount>
参数	方向（up / down）与滚动量（整数）
执行语义	在当前位置执行滚轮滚动
Token 消耗	3–4
示例	sc down 5
底层调用	pyautogui.scroll(±amount)

指令 05：dr（Drag）

字段	内容
操作码	dr
全称	Drag
语法	dr <grid_ref_from> <grid_ref_to>
参数	起点坐标与终点坐标
执行语义	从起点按住左键拖拽至终点并释放
Token 消耗	3–5
示例	dr A1 D8
底层调用	pyautogui.moveTo(x1,y1); pyautogui.drag(dx,dy)
说明	拖拽需要保持按压状态连续，不等同于两个点击

指令 06：hk（Hotkey）

字段	内容
操作码	hk
全称	Hotkey
语法	hk <key1+key2+...>
参数	由 + 连接的键名组合
执行语义	同时按下指定组合键
Token 消耗	2–4
示例	hk Ctrl+S, hk Win+E, hk Alt+F4
底层调用	pyautogui.hotkey(*keys)
说明	该指令是快捷键优先原则的核心载体

指令 07：t（Type）

字段	内容
操作码	t
全称	Type Text
语法	t <text>
参数	任意非换行字符串
执行语义	在当前焦点位置输入文本
Token 消耗	2–N
示例	t Hello World
底层调用	pyautogui.typewrite(text) 或剪贴板粘贴

指令 08：o（Open）

字段	内容
操作码	o
全称	Open
语法	o <path_or_name>
参数	程序名或文件路径
执行语义	启动程序或打开文件
Token 消耗	2–4
示例	o notepad, o C:\report.docx
底层调用	subprocess.Popen() 或 os.startfile()

指令 09：k（Kill）

字段	内容
操作码	k
全称	Kill Process
语法	k <proc_name>
参数	进程名
执行语义	终止指定进程
Token 消耗	2–3
示例	k notepad
底层调用	taskkill 或等效系统调用

指令 10：`w`（Window）

字段	内容
操作码	`w`
全称	Switch Window
语法	`w <window_title>`
参数	窗口标题关键词
执行语义	将目标窗口切换到前台
Token 消耗	2–4
示例	`w Chrome`
底层调用	`win32gui.SetForegroundWindow()` 或等效接口

指令 11：`sl`（Sleep）

字段	内容
操作码	`sl`
全称	Sleep
语法	`sl <seconds>`
参数	正实数
执行语义	暂停执行指定秒数
Token 消耗	2
示例	`sl 2`
底层调用	`time.sleep(n)`
用途	等待启动、加载或动画结束

指令 12：`ld`（Load Library）

字段	内容
操作码	`ld`
全称	Load Dynamic Library
语法	`ld <lib_id>`
参数	库标识符，如 `lib_sys`、`lib_web`
执行语义	将指定工具库加载到当前上下文
Token 消耗	2–3
示例	`ld lib_web`
理论对应	对应 $\forall\pm$ 中的加载态激活

指令 13：`hc`（HOW-code）

字段	内容
操作码	`hc`
全称	HOW-code Create Tool
语法	`hc <tool_name>`
参数	新工具名称
执行语义	触发创造者模型生成工具代码，并进入硬冻结审查
Token 消耗	2–3
示例	`hc img_zip`
安全等级	最高，必须经过 Hard Freeze Gate

指令 14：`mk_lib`（Make Library）

字段	内容
操作码	`mk_lib`
全称	Make Library
语法	`mk_lib <lib_name>`
参数	新库名称
执行语义	创建空的动态库
Token 消耗	2–3
示例	`mk_lib pic`

指令 15：`mv`（Move Tool）

字段	内容
操作码	`mv`
全称	Move Tool to Library
语法	`mv <tool_name> <lib_name>`
参数	工具名与目标库名
执行语义	将指定工具移入目标库
Token 消耗	3–4
示例	`mv img_zip pic`

指令 16：`msg`（Message）

字段	内容
操作码	`msg`
全称	Message to User
语法	`msg <text>`
参数	任意文本
执行语义	向用户输出消息，不产生系统副作用
Token 消耗	2–N
示例	`msg` 任务完成
用途	状态报告、澄清请求、异常提示

3.3. 指令集总表

```
GSS v1.0 — 16 Instructions

MOUSE: c <g> | dc <g> | rc <g> | sc <dir> <n> | dr <g1> <g2>
KEY  : hk <combo> | t <text>
PROC : o <path> | k <name> | w <title>
FLOW : sl <sec>
KNOW : ld <lib>
EVO  : hc <name> | mk_lib <lib> | mv <tool> <lib>
COMM : msg <text>
```

4. 指令集与 Constant-Dalink 公式的关系

4.1. 指令集作为运行时最小常驻动作集

根据 Constant-Dalink 不完备公式：

$$\forall \pm \{(V_1, V_2)^c, V_1, V_2, V, \dots\} \xrightarrow{\text{Check}} \circ V_{use}$$

在操作系统控制场景中，潜在可用知识包括系统 API、各类快捷键、窗口对象、文件系统操作以及应用级菜单与界面元素。这些构成了一个规模较大的知识空间。

但在单步执行时，模型真正需要的不是全部系统知识，而是一个较小的动作子集。GSS 的 16 条指令就是这一运行时动作子集的上界：

$$V_{use} \subseteq \mathcal{O}$$

其中：

$$\mathcal{O} = \{\texttt{c}, \texttt{dc}, \texttt{rc}, \texttt{sc}, \texttt{dr}, \texttt{hk}, \texttt{t}, \texttt{o}, \texttt{k}, \texttt{w}, \texttt{sl}, \texttt{ld}, \texttt{hc}, \texttt{mk_lib}, \texttt{mv}, \texttt{msg}\}$$

并且：

$$|\mathcal{O}| = 16 = \kappa$$

在本文中，这个常数 $\kappa = 16$ 对应 GSS 指令层的认知经济规模。

4.2. 组合态与指令序列

公式中的 $(V_1, V_2)^c$ 表示原子知识的组合。在 GSS 中，它直接对应指令序列。例如，“复制粘贴”可写为：

$$(\texttt{hk Ctrl+C},\ \texttt{hk Ctrl+V})^c$$

这说明 GSS 不需要为每个高层操作定义单独宏指令。只要原子指令集足够完整，高层操作自然可以由组合产生。

4.3. `ld` 与 $\forall\pm$

公式中的 $\forall\pm$ 量化了“加载 / 不加载”的二元状态。在 GSS 中：

- `ld lib_web` 表示将某个库从未加载态切换为加载态；
- 未调用某个库，则其保持未加载状态。

因此，动态库机制是该公式在工具层的直接实现。

4.4. `hc` 与开放知识空间

公式中的省略号 ... 表示知识空间的开放性。`hc` 的作用正是允许系统在运行过程中生成此前尚不存在的新工具项。

这些新工具被创建后并不会自动进入当前最小使用集，而是先经过审查、再进入工具库，并在未来按需通过 `ld` 被激活。

5. `hk` 与快捷键优先原则

5.1. `hk` 的特殊地位

在 16 条指令中，`hk` 具有较高的重要性。原因在于很多桌面操作本来就存在稳定的键盘接口，而不一定需要视觉定位。

本文采用如下原则：

> 能够稳定由快捷键完成的动作，优先使用快捷键。

其主要原因包括：

1. **确定性更高**：快捷键通常对应操作系统或应用程序中较稳定的行为接口；
2. **不依赖空间定位**：`hk Ctrl+S` 不需要坐标信息，而 `c D4` 至少需要目标区域的空间表示；
3. **Token 成本更低**：一条快捷键指令通常比一串点击、等待、搜索与确认动作更短。

5.2. 快捷键覆盖率

对 Windows 常见桌面任务进行粗略分析，可得到表 3。

表 3: 常见任务中的快捷键覆盖率示意

任务类别	快捷键可覆盖比例	代表快捷键
文件操作	$\sim 90\%$	`Ctrl+S`, `Ctrl+O`, `Ctrl+N`, `Ctrl+Z`
窗口管理	$\sim 85\%$	`Alt+Tab`, `Win+D`, `Alt+F4`, `Win+Up`
文本编辑	$\sim 95\%$	`Ctrl+C/V/X/A`, `Ctrl+F`, `Ctrl+H`
系统导航	$\sim 80\%$	`Win+E`, `Win+R`, `Win+I`, `Win+L`
浏览器	$\sim 90\%$	`Ctrl+T/W/L`, `F5`, `Ctrl+Tab`
应用内操作	$\sim 70\%$	与应用实现有关

在典型桌面任务中，快捷键通常可以覆盖大部分交互步骤。这也是 GSS 将 `hk` 置于核心位置的主要原因。

5.3. hk 与其他指令的协同

一个典型 GSS 程序如下：

```
o notepad
sl 1
t Hello World
hk Ctrl+S
sl 0.5
t report.txt
hk Enter
msg 已保存
```

在这一任务中，保存流程完全由快捷键完成，而不依赖鼠标定位。这种模式通常比纯点击式方案更稳定，也更适合小模型生成。

6. 实验设计与性能模型

6.1. 实验目标

本文关注的核心问题是：

> 在 GSS 表示下，7B 级开源模型能否在桌面任务上接近传统大模型方案的可用水平。

6.2. 实验矩阵

设计如下 2×2 对照矩阵，如表 4 所示。

表 4: 实验矩阵

	GSS 框架	传统框架（JSON Schema）
7B 模型	A	B
GPT-4	C	D

其中：

- A：7B + GSS，是本文主要关注的场景；
- B：7B + JSON，用作小模型在传统框架下的对照；
- C：GPT-4 + GSS，作为高能力模型在 GSS 下的参考上界；
- D：GPT-4 + JSON，作为当前常见方案的基线参考。

6.3. 基准任务集

定义 5 个难度递增的桌面任务，如表 5 所示。

表 5: 基准任务集

#	任务	操作步数	涉及指令
T1	打开记事本，输入文本并保存	5–7	`o`, `sl`, `t`, `hk`
T2	打开浏览器，跳转到指定 URL	6–8	`o`, `sl`, `hk`, `t`, `c`
T3	打开文件管理器，创建文件夹并移动文件	8–12	`hk`, `o`, `t`, `c`, `rc`, `sl`
T4	多窗口切换并执行复制粘贴	10–15	`w`, `hk`, `c`, `t`, `sl`
T5	发现缺少工具后调用 HOWcode 生成并注册	5–8 + `hc`	`hc`, `mk_lib`, `mv`, `ld`

6.4. 评估指标

实验中建议使用以下指标：

- **Task Success Rate (TSR)**：任务完全完成的比例；
- **Instruction Accuracy (IA)**：生成指令中合法且正确的比例；
- **Token per Task (TpT)**：单次任务总 Token 消耗；
- **Latency (Lat)**：从任务输入到首条指令输出的延迟；
- **Hallucination Rate (HR)**：非法或不相关指令占总输出的比例。

6.5. 性能预期模型

6.5.1. Token 消耗预期

表 6: Token 消耗预期

场景	TpT（估计）
A: 7B + GSS	120–180
B: 7B + JSON	3000–5000
C: GPT-4 + GSS	120–180
D: GPT-4 + JSON	3000–5000

在该模型下，Token 消耗主要由框架表示决定，而不是由模型规模本身决定。

6.5.2. 任务成功率预期

表 7: 任务成功率预期

场景	T1	T2	T3	T4	T5
A: 7B + GSS	95%	85%	75%	65%	60%
B: 7B + JSON	30%	15%	5%	0%	0%
C: GPT-4 + GSS	98%	95%	90%	85%	80%
D: GPT-4 + JSON	85%	70%	55%	40%	20%

这些数值描述的是本文框架下的预期趋势：GSS 对小模型更友好，而传统结构化框架对小模型更不利。

6.5.3. 幻觉率预期

表 8: 幻觉率预期

场景	幻觉率
A: 7B + GSS	5–10%
B: 7B + JSON	40–60%
C: GPT-4 + GSS	2–5%
D: GPT-4 + JSON	10–20%

在 GSS 中，主要错误通常来自坐标选择、层级选择或步骤判断；在 JSON 框架中，格式错误本身就是额外误差来源。

6.5.4. 延迟预期

表 9: 首条指令延迟预期

场景	首条指令延迟
A: 7B 本地 + GSS	0.3–1.0 秒
B: 7B 本地 + JSON	2–5 秒
C: GPT-4 云端 + GSS	1–3 秒
D: GPT-4 云端 + JSON	3–8 秒

在端侧场景中，GSS 的低上下文长度有利于降低首条动作延迟。

6.6. 理论解释

7B + GSS 与 GPT-4 + JSON 之间可能出现接近结果，其原因不在于小模型本身跨越了全部能力差距，而在于任务表示被显著简化，如表 10 所示。

表 10: JSON 框架与 GSS 框架对模型要求的差异

维度	JSON 框架	GSS 框架
输出形式	嵌套结构化 JSON	短文本原子指令
上下文规模	大量 Schema 与说明	简短指令字典
动作空间	开放结构值	固定操作码 + 受限参数
单步 Token 链长度	较长	较短

因此，GSS 的作用不是直接提高模型能力，而是降低桌面任务在语言生成层上的表达难度。

7. 端侧部署架构

7.1. 部署拓扑

```
+-----------------------------------+
|          Edge Device              |
|  +-----------+   +-------------+ |
|  |  7B Model |<->|   GSS-OS    | |
|  |  (GGUF)   |   |   Runtime   | |
|  |  4-bit    |   | parser.py   | |
|  |  quant    |   | executor.py | |
|  +-----------+   | grid.py     | |
|                  | gate.py     | |
|                  +-------------+ |
|                       ^          |
|                       |          |
|                 +-----------+    |
|                 | OS Desktop|    |
|                 +-----------+    |
+-----------------------------------+
```

7.2. 硬件需求

表 11: 端侧部署硬件需求

组件	最低要求	推荐配置
CPU	8 核 x86/ARM	12 核以上
RAM	8 GB	16 GB
GPU	可无	6 GB VRAM（4-bit 量化推理）
存储	10 GB	20 GB
网络	不要求	不要求

7.3. 推理引擎

表 12: 推理引擎　配性

引擎	特点	适配性
`llama.cpp`	CPU / GPU 混合推理	高
`ollama`	部署封装较友好	较高
`vLLM`	吞吐优先	更适合服务端

由于 GSS 的单次输入与输出长度较短，端侧推理的延迟压力相对较小。

7.4. 隐私与安全意义

端侧部署带来几个直接特性：

1. 屏幕内容、文件名与操作序列不必离开本机；
2. 在无网络环境下仍可运行；
3. 不产生按 Token 计费的云端成本；
4. 推理延迟不受网络波动影响。

这些特性在企业内网、政府办公、医疗信息处理等场景中具有实际意义。

8. 与现有框架的工程比较

8.1. 系统级对比

表 13: GSS-OS 与几类常见框架的工程比较

维度	AutoGPT	LangChain Agent	Anthropic CU	GSS-OS
最小模型	GPT-4	GPT-3.5+	Claude 3.5	7B 开源模型可用
单次 Token	5000+	3000+	7000+	~ 150
调用格式	JSON	JSON	自然语言 + 坐标	原子文本
视觉依赖	无或插件	无或插件	全屏截图	Grid 坐标
端侧部署	较弱	较弱	较弱	强
离线运行	较弱	较弱	较弱	强
自进化	弱	弱	弱	支持 HOW-code
安全审查	弱或无	弱	弱	Hard Freeze
指令集规模	不固定	不固定	不固定	16 条固定

8.2. 适用范围

GSS-OS 的目标并不是替代所有 Agent 框架。它更适用于如下场景：

在操作系统桌面控制这一特定任务域内，以较小模型、较少 Token 和较低延迟完成稳定动作生成。

对于 API 编排、数据库操作或云服务调度，JSON Schema 仍然是合理的表示。GSS 主要关注的是鼠标、键盘、窗口和本地桌面交互这一层。

9. 局限性与结论

9.1. 局限性

本文仍有若干局限：

1. **跨平台支持**：当前实现主要面向 Windows。macOS 与 Linux 的快捷键体系和窗口控制接口仍需适配。
2. **长链任务规划**：当任务链较长时，7B 模型的规划稳定性可能下降，需要进一步引入分步规划或状态回顾机制。
3. **动态界面变化**：对于频繁弹窗、布局变化明显或存在广告干扰的应用，单轮动作生成可能不足，需要更强的观察–重规划循环。
4. **无快捷键场景**：某些纯图形化界面中快捷键支持较弱，此时系统将更多依赖 Grid 与鼠标类指令。

9.2. 后续方向

后续工作可包括：

1. 为 macOS 与 Linux 建立快捷键映射与执行适配；
2. 引入轻量观察–重规划循环；
3. 在保持 κ 可控前提下评估是否需要极少量指令扩展；
4. 建立统一的 GSS 桌面任务基准集；
5. 在 7B 模型上使用 GSS 指令数据进行监督微调。

9.3. 结论

本文给出了 GSS-OS 的 16 条原子指令集，并从设计原则、理论解释、工程实现和实验设计四个方面进行了整理。

核心结论可概括如下：

表 14: 本文主要结论

#	结论
1	16 条原子指令可覆盖常见的人机物理交互通道
2	指令集规模 $\|\mathcal{O}\| = 16$ 对应 Constant-Dalink 框架下的认知经济常数 κ
3	快捷键优先原则可显著减少对空间定位的依赖
4	在 GSS 表示下，7B 模型的桌面任务可用性有望接近传统大模型方案
5	单次任务 Token 成本可维持在约 150 左右
6	GSS-OS 适合端侧、离线和隐私敏感场景的桌面控制

本文的中心观点是：在桌面控制任务中，模型能力并不是唯一瓶颈，任务表示方式同样重要。

当系统要求模型生成较长结构化输出、处理较大上下文并隐式恢复复杂操作结构时，小模型往往表现较差；而当桌面动作被压缩为一组简短、可组合、可预测的原子指令时，任务本身会更接近小模型可稳定处理的形式。

从这个意义上说，GSS 的作用不是单纯“让模型更强”，而是将桌面控制问题改写为更适合端侧文本模型处理的形式。

参考文献

[1] OpenAI. Function Calling and Other API Updates. 2023.

[2] Harrison Chase. LangChain: Building Applications with LLMs through Composability. 2022.

[3] Significant Gravitas. AutoGPT: An Autonomous GPT-4 Experiment. 2023.

[4] Anthropic. Developing a Computer Use Model. 2024.

[5] David A. Patterson and John L. Hennessy. Computer Organization and Design: The RISC-V Edition. 2017.

[6] Hugo Touvron et al. LLaMA: Open and Efficient Foundation Language Models. 2023.

[7] Tim Dettmers et al. QLoRA: Efficient Finetuning of Quantized Language Models. *NeurIPS*, 2023.

[8] Tom B. Brown et al. Language Models are Few-Shot Learners. *NeurIPS*, 2020.

[9] Claude E. Shannon. A Mathematical Theory of Communication. 1948.

[10] GSS-OS Internal Notes. Constant-Dalink Incompleteness Formula and Cognitive Compression. 2025.

A. 完整 Prompt 模板示例

以下是 prompt.py 中 SYSTEM 模板的一个最小化示例：

```
GSS-OS v1.0. You are a desktop operator. Output ONLY commands.
Commands:
c <g> click | dc <g> dblclick | rc <g> rclick
sc <dir> <n> scroll | dr <g1> <g2> drag
hk <keys> hotkey | t <text> type
o <path> open | k <name> kill | w <title> window
sl <s> sleep | ld <lib> load
hc <n> create | mk_lib <n> mklib | mv <t> <l> move
msg <text> message
Grid: l4 (16x16). Prefer hk over c.
```

该模板在较小 Token 预算内包含了：

- 16 条指令定义；
- 当前 Grid 层级；
- 一条核心策略提示。

B. 端到端任务执行示例

任务：

“打开记事本，输入 Hello World，保存为 test.txt”

模型输出：

```
o notepad
sl 1
t Hello World
hk Ctrl+S
sl 0.5
t test.txt
hk Enter
msg 已保存为 test.txt
```

在该示例中：

- 输入部分主要由指令模板与任务描述组成；
- 输出部分是 8 条原子指令；
- 整个任务保持在较低 Token 范围内。

C. Constant-Dalink 公式在本文中的映射

$$\forall \pm \{(V_1, V_2)^c, V_1, V_2, V, \dots\} \xrightarrow{\text{Check}} \circ V_{use}$$

其对应关系如表 15 所示。

表 15: Constant-Dalink 公式与本文的映射

公式组件	本文中的映射
$\forall\pm$	`ld` 所表示的加载 / 不加载控制
$(V_1, V_2)^c$	指令序列组合
V_1, V_2	16 条原子指令
V	动态库中的工具项
$\dots$	`hc` 生成的未来工具
$\xrightarrow{\text{Check}}$	`parser.py` 过滤、校验与审查流程
$\circ V_{use}$	运行时 prompt 中实际激活的最小动作集
$\kappa = 16$	指令集规模对应的认知经济常数